UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

Introduction to Deep Learning

Single-Layer Neural Networks



Acknowledgements

Slides of this lecture make use of / are based on:

- Andreas Geiger (University of Tübingen): Deep Learning (lecture website)
  Here: slide decks 1 and 2 (Introduction, Computation Graphs)
- All plots in this lecture: own figures (`fig/03/make_figures.py`)

Agenda



- 3.1 Linear regression
- 3.2 Gradient descent
- 3.3 Perceptron
- 3.4 Logistic regression
- 3.5 Computation graphs

Notation (Recap from Lecture 2)

Dataset with n instances	$\mathcal{D} = \{(\boldsymbol{x}_1, y_1), ..., (\boldsymbol{x}_n, y_n)\}$
Feature vector of instance i	$\boldsymbol{x}_i = \left(x_{i,1}, ..., x_{i,k}\right)$, k features
Label / target of instance i	y_i
Model with weights $\boldsymbol{w} = (w_0, ..., w_k)$	$f_{\boldsymbol{w}}$
Prediction	$\hat{y}_i = f_{\boldsymbol{w}}(\boldsymbol{x}_i)$
Loss of a single prediction	$\mathcal{L}(\hat{y}_i, y_i)$
Total error / loss on the training data	$E(\boldsymbol{w}) = \sum_{i=1}^{n} \mathcal{L}(\hat{y}_i, y_i)$

In this lecture, “error function” E and “loss function” $\mathcal{L}$ mean the same idea: E sums the loss over all training instances.

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

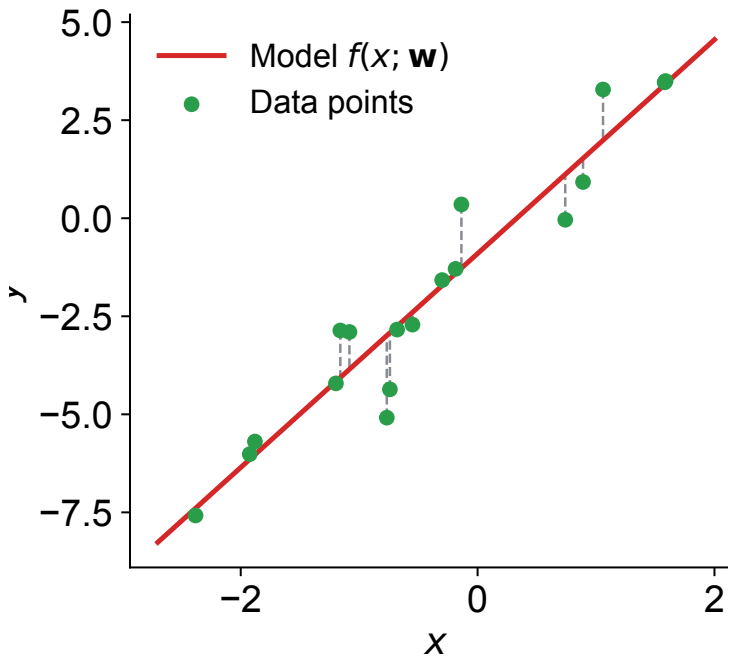
3.1 Linear Regression



Linear Regression

- Predict a continuous y for new input data $\mathbf{x}$.
- We write each training example as $(\mathbf{x}_i, y_i) \in \mathcal{D}$.
- $\mathbf{x}_i$ can be multidimensional
- Functional form (straight line / hyperplane):

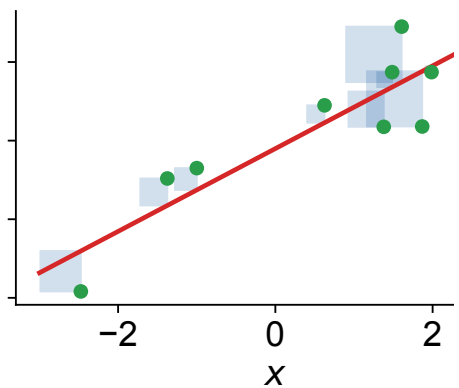
$$\hat{y}_i = w_0 + w_1 \cdot x_{i,1} + ... + w_k \cdot x_{i,k}$$



Error Function

- The **error function** $E(w)$ measures the displacement along the y dimension between the data points and the model predictions
- We use here (as it is common) the **least squares error**
- Data points (green) and model $f(x; w)$ (red) specified by the parameters w

$$
\begin{aligned}
E(\boldsymbol{w}) &= \sum_{i=1}^{n} \left(f(\boldsymbol{x}_i, \boldsymbol{w}) - y_i\right)^2 \\
&= \sum_{i=1}^{n} \left(\hat{y}_i - y_i\right)^2 \\
&= \sum_{i=1}^{n} \left(\left(w_0 + w_1 \cdot x_{i,1} + \ldots + w_k \cdot x_{i,k}\right) - y_i\right)^2
\end{aligned}
$$



Matrix Notation

- Compact way of writing linear regression: Matrix notation

One row: one instance →

$$\underbrace{\begin{pmatrix} \boxed{1} & \boxed{2} & \boxed{3} \\ 1 & 4 & 5 \\ 1 & 6 & 7 \end{pmatrix}}_{X} \cdot \underbrace{\begin{pmatrix} w_0 \\ w_1 \\ w_2 \end{pmatrix}}_{w} = \begin{pmatrix} \hat{y}_0 \\ \hat{y}_1 \\ \hat{y}_2 \end{pmatrix}$$

Add a constant column! Original Dataset

- This gives us the same computations!
- Example for the first instance: $\hat{y}_0 = 1 \cdot w_0 + 2 \cdot w_1 + 3 \cdot w_2$
- Optimization in matrix notation: $\min_w \|Xw - y\|_2^2$
- In general: $f(\boldsymbol{w}, \boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$

Minimizing the Error Function

- For this special case, the best parameters **w** (the parameters that optimize the error function) can be computed in closed form:

$$w = (X^\top X)^{-1} X^\top y$$

- So-called **normal equation**; $(X^\top X)^{-1} X^\top$ is the **Moore-Penrose pseudo-inverse** of X
- Overall complexity: $O(np^2 + p^3)$ (n instances, p weights)

- Solution is exact (up to numerical precision)
- Gets costly for large datasets!

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

3.2 Gradient Descent

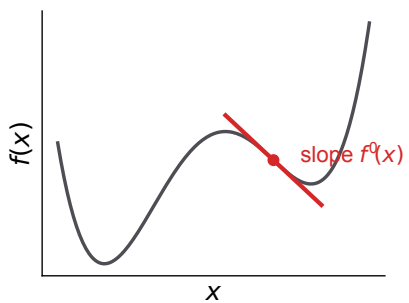


Gradient Descent

Gradient Descent

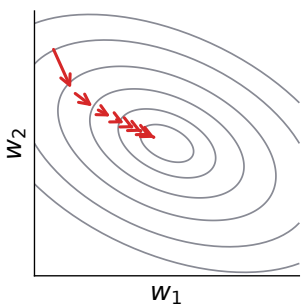
Gradient

= The vector of all partial derivatives of a function



Descent

= Finding a way towards the minimum of the function



Gradient Descent

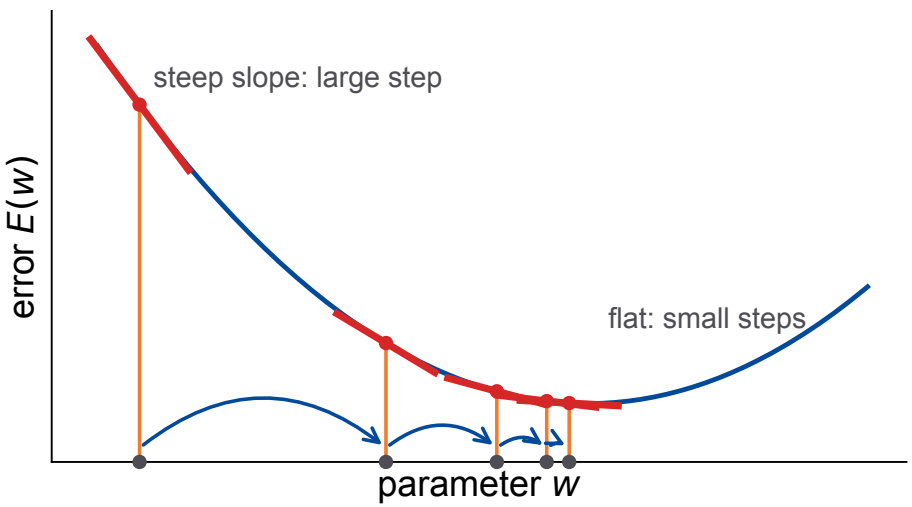
UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- Gradient descent is a standard solution for (m)any optimization problems
- Very often used for fitting parameters to data
- Improve solution step-by-step, reducing the error in each step:
- Gradient gives direction of steepest *ascent*
- A simple way to minimize a (differentiable) function $f(x)$:
  1. Compute the gradient ∇f (derivative function)
  2. Start at some (random) point x_0
  3. Evaluate $\nabla f(x_t)$
  4. Take a step in the reverse direction of the gradient: $x_{t+1} = x_t - \alpha \nabla f(x_t)$
     α: “learning rate” or “step size”
  5. Repeat 3-4 until converged (or a max number of iterations)

What We Optimize

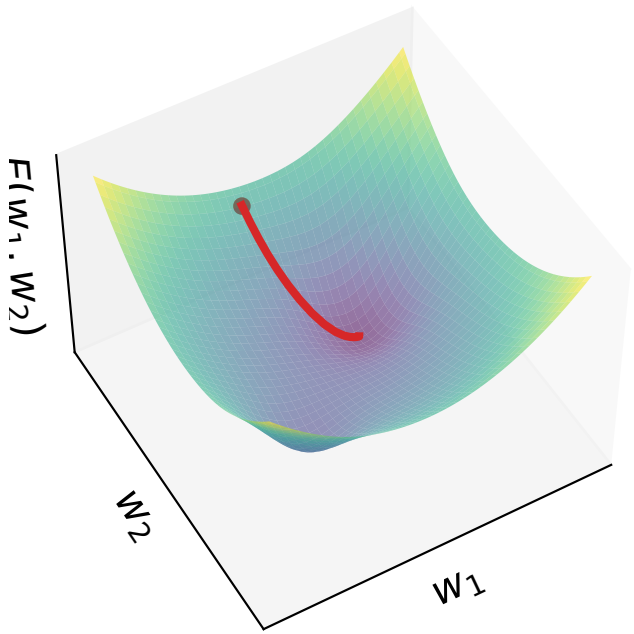
- What we are optimizing here is the function $f(x)$
- When applying this to regression / ML, x is (typically) the weights w!
- NOT the input data x

- Step size $\alpha \ |\nabla E|$ shrinks automatically where the error gets flat



Gradient Descent with Two Parameters

- Here with the error depending on 2 parameters
- The path follows the steepest descent on the error surface
- In practice the error depends on k parameters, thus would have to be visualized in k+1 dimensional space!

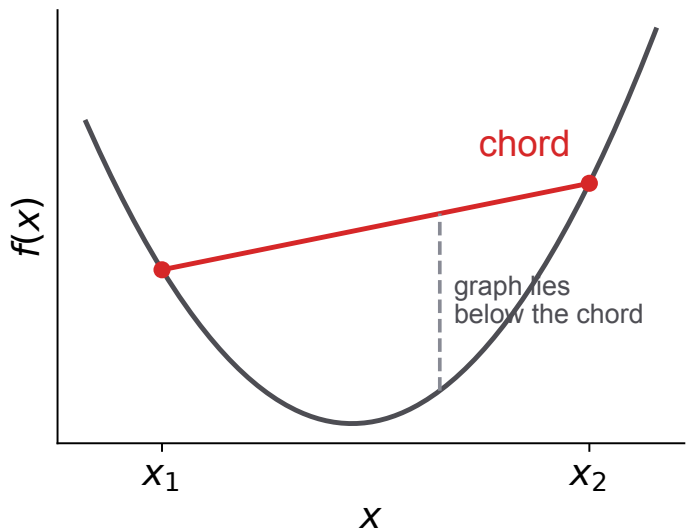


Convergence

- If α is small enough, we will reduce the error
- But do we end up in a global minimum?

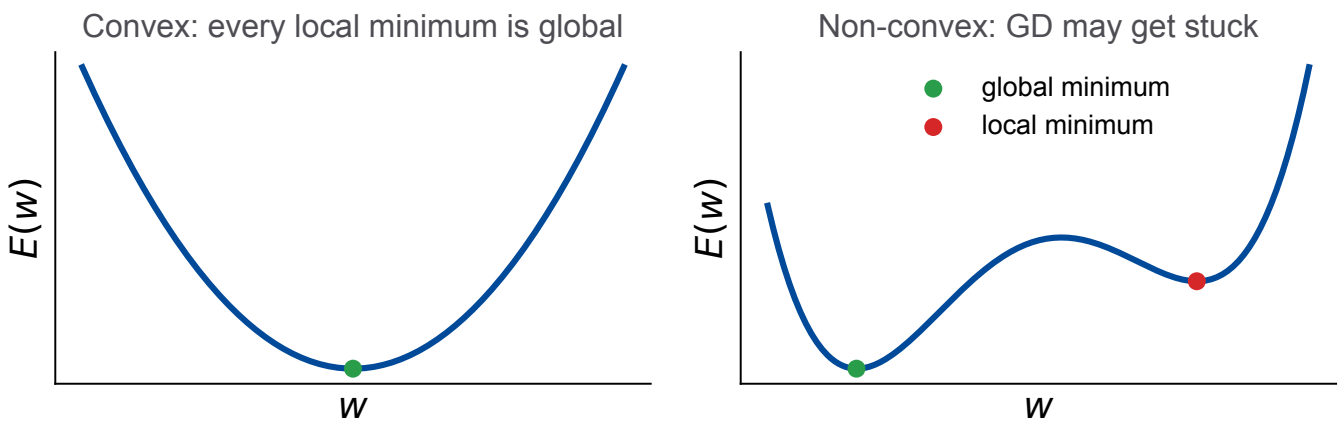
- In general: No
  - Gradient descent finds any (local) minimum
  - Not necessarily the global one

- For convex functions,
  low “enough” learning rate: yes



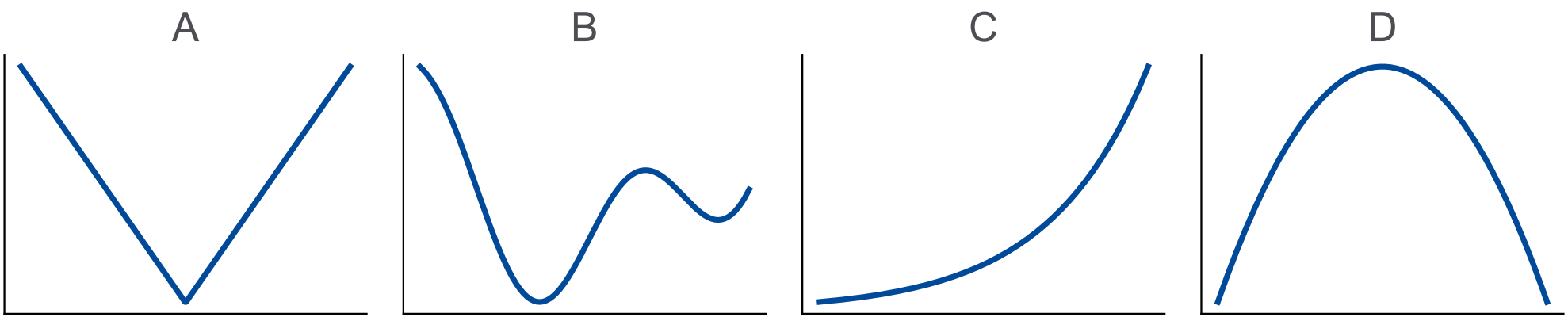
- Convex function: the line segment between any two points of the graph lies on or above the graph

Convex vs. Non-Convex

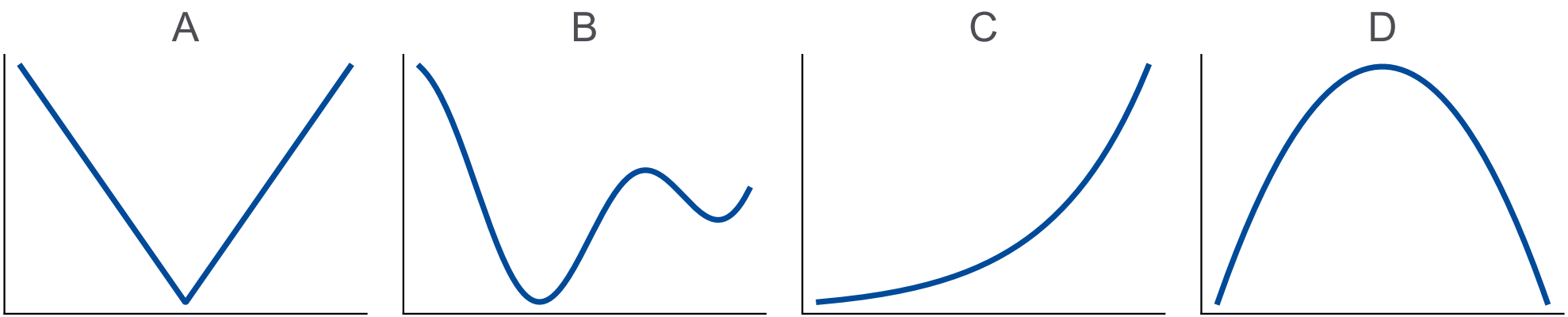


- Linear regression (squared error): convex → GD finds the global minimum
- Neural networks: highly non-convex – but GD still works surprisingly well

Quiz: Which Functions Are Convex?



Quiz: Which Functions Are Convex?



A $|x|$: convex (not differentiable at 0) B: not convex (local minima) C e^x: convex D $-x^2$: concave, not convex

Linear Regression with GD

Cost Function:

$$E(\boldsymbol{w}) = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \sum_{i=1}^{n} \left(\left(w_0 + w_1 \cdot x_{i,1} + ... + w_k \cdot x_{i,k} \right) - y_i \right)^2$$

Gradient:

$$\frac{\partial E(\boldsymbol{w})}{\partial w_0} = 2 \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

$$\frac{\partial E(\boldsymbol{w})}{\partial w_1} = 2 \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot \frac{\partial}{\partial w_1} (\hat{y}_i - y_i) = 2 \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot x_{i,1}$$

$$...$$

$$\frac{\partial E(\boldsymbol{w})}{\partial w_k} = 2 \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot \frac{\partial}{\partial w_k} (\hat{y}_i - y_i) = 2 \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot x_{i,k}$$

The Gradient in Matrix Notation

- We can also determine the gradient in matrix notation!

$$
\begin{aligned}
\nabla_w E(w) &= \nabla_w \, \|Xw - y\|_2^2 && \text{Gradient in Matrix notation} \\
&= \nabla_w (Xw - y)^\top (Xw - y) && \text{Express the squared norm as the inner product} \\
&= \nabla_w (w^\top X^\top X w - 2 w^\top X^\top y + y^\top y) && \text{Expand} \\
&= 2X^\top X w - 2X^\top y && \text{Compute gradients}
\end{aligned}
$$

- Setting the gradient to zero gives the closed-form solution: $X^\top X w = X^\top y$

Update Rule

- Start with a random value for $w_0, \dots, w_k$
- In each step update the weights $w_0, \dots, w_k$:

$$w_0^{\text{new}} = w_0^{\text{old}} - \alpha \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

$$w_1^{\text{new}} = w_1^{\text{old}} - \alpha \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot x_{i,1}$$

$$\dots$$

$$w_k^{\text{new}} = w_k^{\text{old}} - \alpha \cdot \sum_{i=1}^{n} (\hat{y}_i - y_i) \cdot x_{i,k}$$

Predicted outcome

True outcome

Worked Example

- Data: $(x, y) \in \{(1, 2), (2, 4)\}$, model $\hat{y} = wx$ (no bias)
- $E(w) = (w - 2)^2 + (2w - 4)^2$
- $E'(w) = 2 \sum_i (wx_i - y_i)x_i = 10w - 20$
- Start $w_0 = 0$, learning rate $\alpha = 0.05$

t	w_t	$E'(w_t)$	$E(w_t)$
0	0	−20	20
1	1	−10	5
2	1.5	−5	1.25
3	1.75	−2.5	0.31
⋮	$\to 2$	$\to 0$	$\to 0$

Quiz: What happens with $\alpha = 0.25$?

Worked Example

- Data: $(x, y) \in \{(1, 2), (2, 4)\}$, model $\hat{y} = wx$ (no bias)
- $E(w) = (w - 2)^2 + (2w - 4)^2$
- $E'(w) = 2\sum_i (wx_i - y_i)x_i = 10w - 20$
- Start $w_0 = 0$, learning rate $\alpha = 0.05$

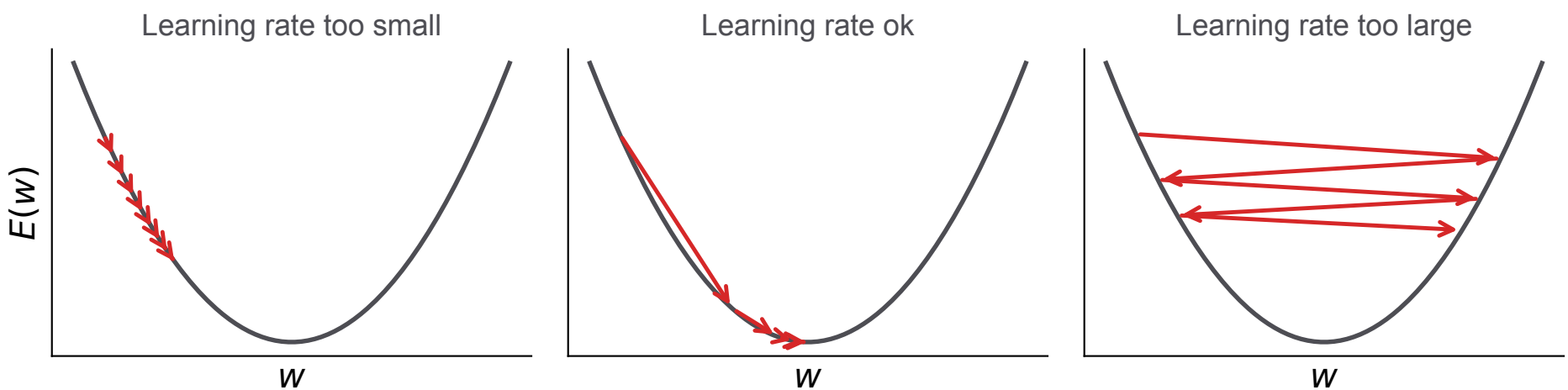
t	w_t	$E'(w_t)$	$E(w_t)$
0	0	−20	20
1	1	−10	5
2	1.5	−5	1.25
3	1.75	−2.5	0.31
⋮	$\to 2$	$\to 0$	$\to 0$

Quiz: What happens with $\alpha = 0.25$?

$w : 0 \to 5 \to -2.5 \to 8.75 \to \ldots$ Each step overshoots the minimum $w = 2$ by more than before: GD diverges.

How to Pick the Step Size α?

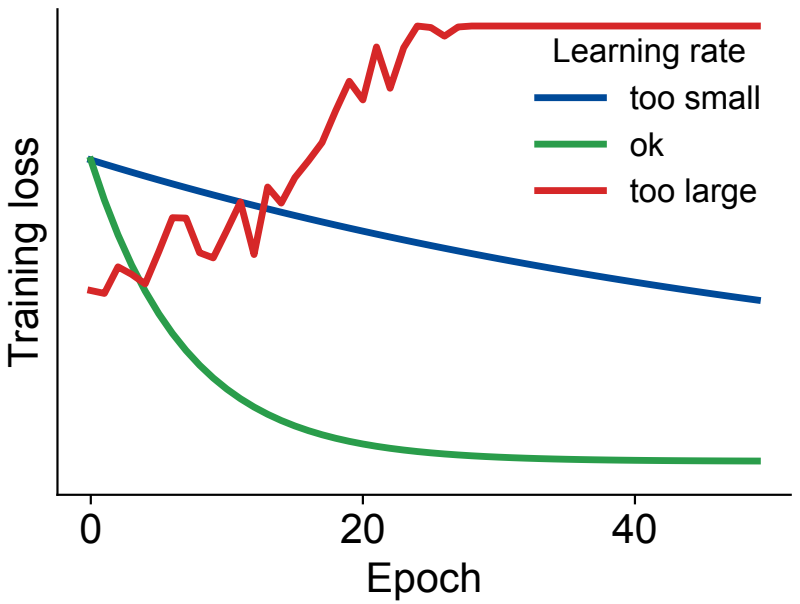
- α is also called the **learning rate**
- α too small:
  - very slow convergence
  - may stop (max. iterations) far from the minimum
- α too large:
  - Will "overshoot"
  - Might not converge at all



Learning Rate in Practice

- Always plot the **training loss over epochs**
  - decreases slowly → increase α
  - jumps around or increases → decrease α
- Typical starting values: 10^{-3} … 10^{-1} (depends on optimizer and model)
- Common: try values on a log scale (0.001, 0.01, 0.1)
- Outlook: **learning-rate schedules** (decay, warm-up)

- Try it: `exercise/gradient_demo.html` (Gradient Explorer)



Stochastic Gradient Descent

- Computing the gradient requires a sum over all instances
  ➔ can be slow for large datasets!
- Approximate the gradient with just one random example
- More often: **mini-batch:** Use a small subset of instances, and compute the error/gradient for those instances

While *not converged or maximum_iterations_reached*:

- Draw random batch from training set
- Compute the approximated gradient $\nabla f(w_t)$ with the batch
- New parameters $w_{t+1} = w_t - \alpha \nabla f(w_t)$ (learning rate α)

Epoch and Batch Size

- The **batch size** is a hyperparameter
  - determines how many training examples you look at until you update your parameters (weights w)
  - Often limited by available memory (GPU/CPU limits); typical: 16 … 512 (powers of 2)
- An **epoch**: complete pass over the training data (each example used once)
  - Number of epochs: hyperparameter, dependent on task/data
- An **iteration** (step): one parameter update = one mini-batch

Quiz: $n = 10\ 000$ training examples, batch size 32: how many updates per epoch?

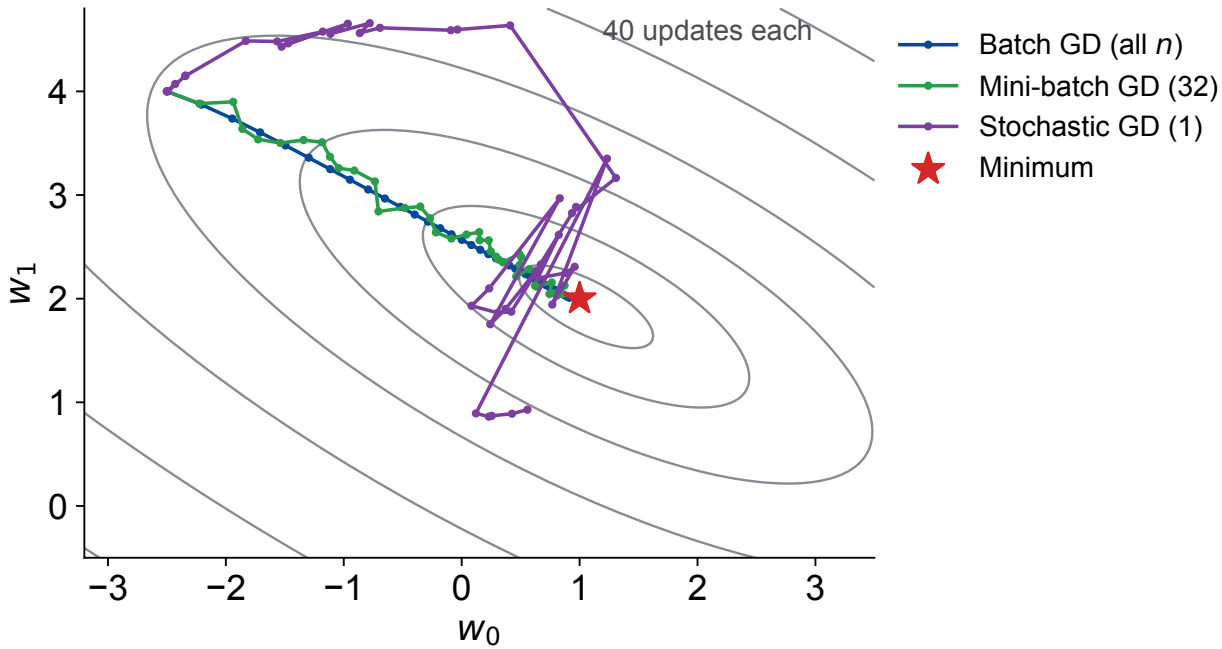
Epoch and Batch Size

- The **batch size** is a hyperparameter
  - determines how many training examples you look at until you update your parameters (weights w)
  - Often limited by available memory (GPU/CPU limits); typical: 16 … 512 (powers of 2)
- An **epoch**: complete pass over the training data (each example used once)
  - Number of epochs: hyperparameter, dependent on task/data
- An **iteration** (step): one parameter update = one mini-batch

Quiz: $n = 10\ 000$ training examples, batch size 32: how many updates per epoch?

> $\lceil 10\ 000/32 \rceil = 313$ updates (312 full batches + one batch of 16) – with full-batch GD it would be 1 update.

Comparison



Linear regression on 500 synthetic points; same start and learning rate for all three

Why the Noise Is Not Only Bad

- Mini-batch gradients are noisy estimates of the true gradient – but:
  - **Cheap:** cost per update independent of $n \rightarrow$ many more updates per epoch
  - **Redundancy:** large datasets contain many similar examples; a small batch already points in a good direction
  - **Escaping:** noise helps to leave saddle points and flat regions
  - **Generalization:** small-batch SGD tends to find “flat” minima that generalize better (implicit regularization)
- Near the end of training, the noise prevents exact convergence → reduce the learning rate

Keskar et al.: On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. ICLR, 2017

Outlook

- Later in the lecture: Improvements on this algorithm:
  - Momentum-based algorithms
  - RMSProp
  - Adam

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

3.3 Perceptron



Binary Outputs

- In Linear Regression, the outputs can be any number
- If we want to build a classification algorithm for binary classification, we have to predict either positive or negative (1 or −1)
- Perceptron:
  - Binary classifier, labels $y \in \{-1, 1\}$ (logistic regression later: $y \in \{0, 1\}$)
  - Linear Weighted Function + Decision Rule
- Analogy: Neuron that “fires” if it gets “enough” input

$$f(\boldsymbol{w}, \boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x}$$

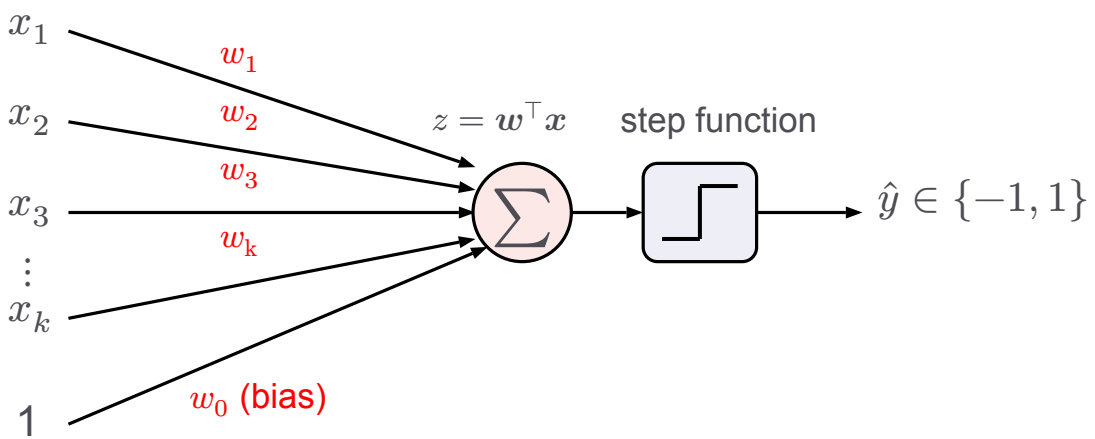
Linear Regression:

$$f(\boldsymbol{w}, \boldsymbol{x}) = \begin{cases} 1 & \boldsymbol{w}^\top \boldsymbol{x} \geq 0 \\ -1 & \boldsymbol{w}^\top \boldsymbol{x} < 0 \end{cases}$$

Perceptron:

Here: $\boldsymbol{x} = [1, x]^\top \Rightarrow f(\boldsymbol{x}, \boldsymbol{w}) = w_0 + w_1 x$

Simple Perceptron



- Weighted sum of the inputs (plus bias), followed by a step function

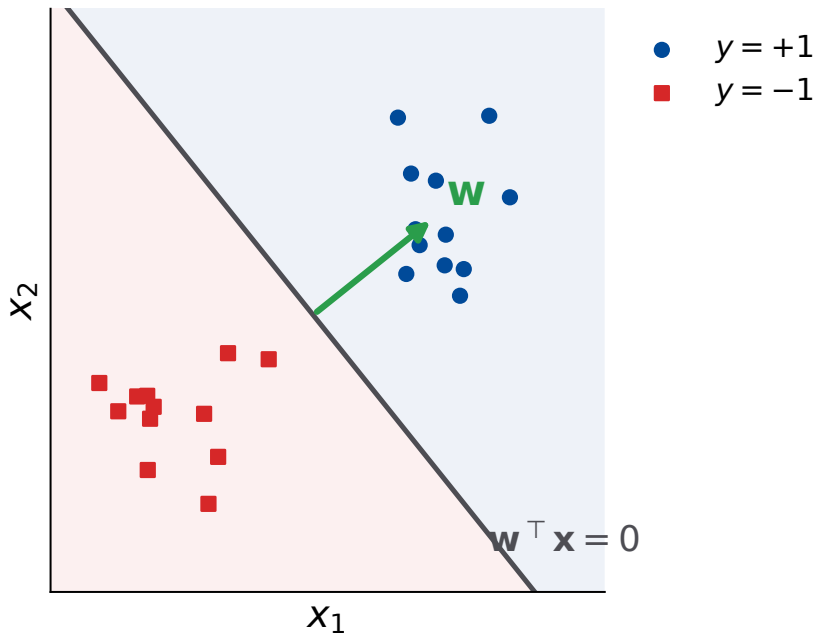
A Bit of History

- **1943** McCulloch & Pitts: model of a neuron as a threshold unit (fixed weights)
- **1958** Rosenblatt: the **perceptron** – weights **learned** from data
  Mark I Perceptron: hardware for image recognition (20 × 20 photocells)
- **1969** Minsky & Papert, *Perceptrons*: a single perceptron cannot learn simple functions like XOR
  → interest in neural networks dropped for years
- **1986** Rumelhart, Hinton & Williams: backpropagation for **multi-layer** networks
  → lecture 4

Rosenblatt: The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 1958

Geometric View

- Decision boundary: $\boldsymbol{w}^\top \boldsymbol{x} = 0$
  $\rightarrow$ a line (2D), a plane (3D), a hyperplane in general
- $\boldsymbol{w}$ (without w_0) is the **normal vector** of the boundary
- The bias w_0 shifts the boundary away from the origin
- Points on the side $\boldsymbol{w}$ points to: $\boldsymbol{w}^\top \boldsymbol{x} > 0 \rightarrow$ class $+1$



Training of Perceptrons

- GD/SGD not applicable! Threshold function is not differentiable
- Update rule: if we make an error for one instance i
  - Adapt the weights in the “right” direction
  - The higher the input, the larger the adjustment

$$w_0^{\text{new}} = w_0^{\text{old}} - \alpha \cdot (\hat{y}_i - y_i)$$

$$w_1^{\text{new}} = w_1^{\text{old}} - \alpha \cdot (\hat{y}_i - y_i) \cdot x_{i,1}$$

$$\dots$$

$$w_k^{\text{new}} = w_k^{\text{old}} - \alpha \cdot (\hat{y}_i - y_i) \cdot x_{i,k}$$

Predicted outcome

True outcome

Perceptron Update: Example

- Weights $\boldsymbol{w} = (w_0, w_1, w_2) = (0, 1, -1)$, learning rate $\alpha = 0.5$
- Instance $\boldsymbol{x} = (1, 1, 2)$ (first entry: constant 1 for the bias), label $y = +1$

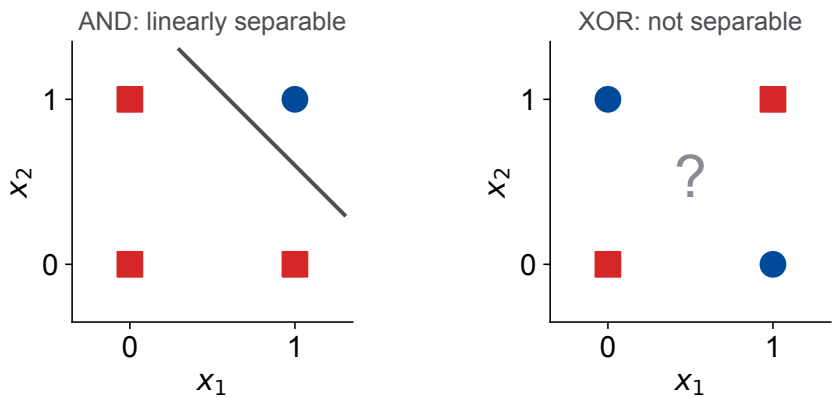
- Prediction: $\boldsymbol{w}^\top \boldsymbol{x} = 0 + 1 - 2 = -1 < 0 \;\Rightarrow\; \hat{y} = -1$ × wrong
- Update: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha(\hat{y} - y)\boldsymbol{x} = (0, 1, -1) - 0.5 \cdot (-2) \cdot (1, 1, 2) = (1, 2, 1)$
- New prediction: $\boldsymbol{w}^\top \boldsymbol{x} = 1 + 2 + 2 = 5 \geq 0 \;\Rightarrow\; \hat{y} = +1$ ✓

- Correctly classified instances: $\hat{y} - y = 0 \rightarrow$ no update
- With labels ± 1, the rule is often written as $\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha y \boldsymbol{x}$ (for misclassified $\boldsymbol{x}$)

Convergence and Limits

- It has (very early!) been shown that this learning algorithm finds a solution for every problem that the perceptron can solve: if the data are **linearly separable**, it converges after a finite number of updates (Novikoff, 1962)
- [No proof here]

- But: many problems are not linearly separable, e.g., XOR
  → needs several layers (lecture 4)



UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

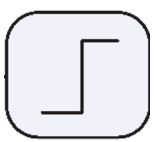
3.4 Logistic Regression



Two Deficits of the Perceptron

In this form, the perceptron has two main deficits (among others…):

1. Learning for Perceptrons works – but it is not a “principled” method like gradient descent
   If we try to use gradient descent, the gradient is actually zero or undefined for the step function!

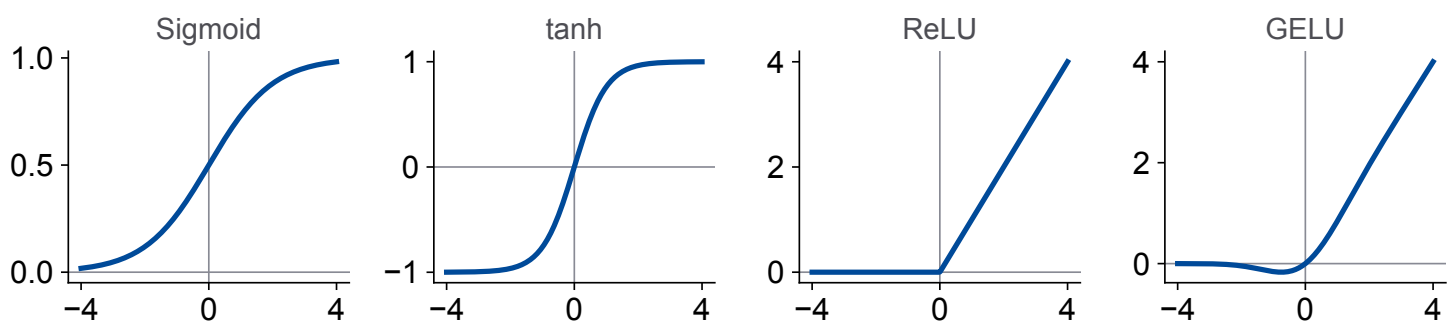


2. We only get a yes – no answer for the class.
   Sometimes, it would be nice if we could get more details, e.g., a certainty/probability for a class (soft classification)

Activation Functions

- To address these issues, we can replace the step-function in the perceptron with other functions: **activation functions**
- Very common ones are:
  - Sigmoid-types: **logistic function (sigmoid)**, hyperbolic tangent (tanh)
  - Partially linear: ReLU, Leaky ReLU, GELU

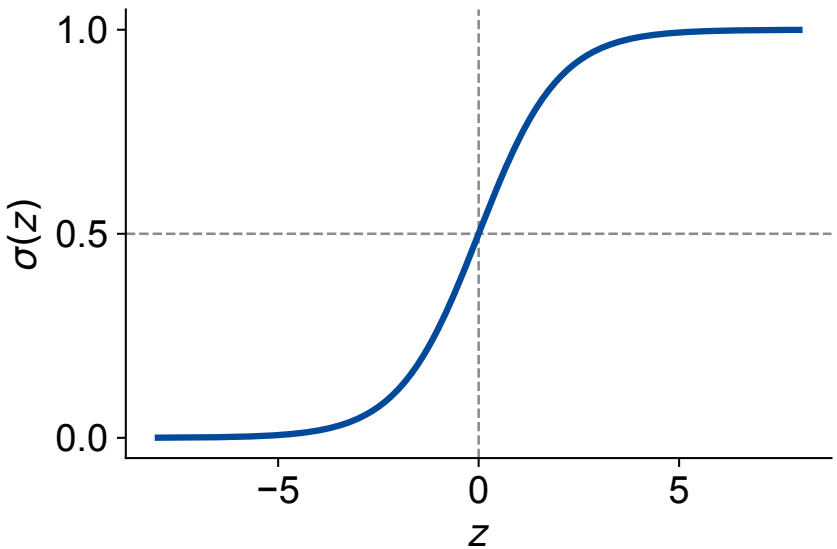
We will learn more about other activation functions later on!



Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}$$

- Squeezes any input into (0, 1)
- $\sigma(0) = 0.5, \;\; \sigma(-z) = 1 - \sigma(z)$
- Smooth and differentiable everywhere
- Interpretation: probability of the positive class



Derivative of the Sigmoid Function

$$\sigma(z) = \frac{1}{1+e^{-z}} = (1+e^{-z})^{-1}$$

$$\frac{d\sigma(z)}{dz} = -(1+e^{-z})^{-2} \cdot (-e^{-z})$$

Chain Rule

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$

Simplify

Remember:

$$1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z) \cdot (1-\sigma(z))$$

Logistic Regression Model

- Replace the step function by the sigmoid:

$$\hat{y} = f_{\boldsymbol{w}}(\boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x}) = P(y = 1 \mid \boldsymbol{x})$$

- Labels $y \in \{0, 1\}$; output is a probability (soft classification)
- Decision rule: predict class 1 if

$$\hat{y} \geq 0.5 \;\Leftrightarrow\; \boldsymbol{w}^\top \boldsymbol{x} \geq 0$$

- $\rightarrow$ Same **linear** decision boundary as the perceptron – but now with a probability and a differentiable model

Probabilistic Predictions

- With sigmoids, we can model uncertainty in the outcome
- But… not every certainty score is a probability → we need calibration!

- If we interpret predictions as probabilities,
  how should we choose the weights of our prediction model?

Such that the overall likelihood of all predictions coming true gets maximized

Maximum Likelihood Estimator

- We search

$$\hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmax}} \prod_{i=1}^{N} p_{\text{model}}(y_i \mid \boldsymbol{x}_i, \boldsymbol{w})$$

$$\Leftrightarrow \hat{\boldsymbol{w}} = \underset{\boldsymbol{w}}{\operatorname{argmax}} \sum_{i=1}^{N} \log p_{\text{model}}(y_i \mid \boldsymbol{x}_i, \boldsymbol{w})$$

- Assume that for each instance, we make a guess with the predicted probability of the model
- What is the likelihood of “guessing” correctly for one instance?

$$p_{\text{model}}(y \mid \boldsymbol{x}, \boldsymbol{w}) = \hat{y}^{y}(1-\hat{y})^{(1-y)}$$

This is a compact way of writing $p_{\text{model}}(y \mid \boldsymbol{x}, \boldsymbol{w}) = \begin{cases} \hat{y} & \text{if } y = 1 \\ 1-\hat{y} & \text{if } y = 0 \end{cases}$

Binary Cross Entropy

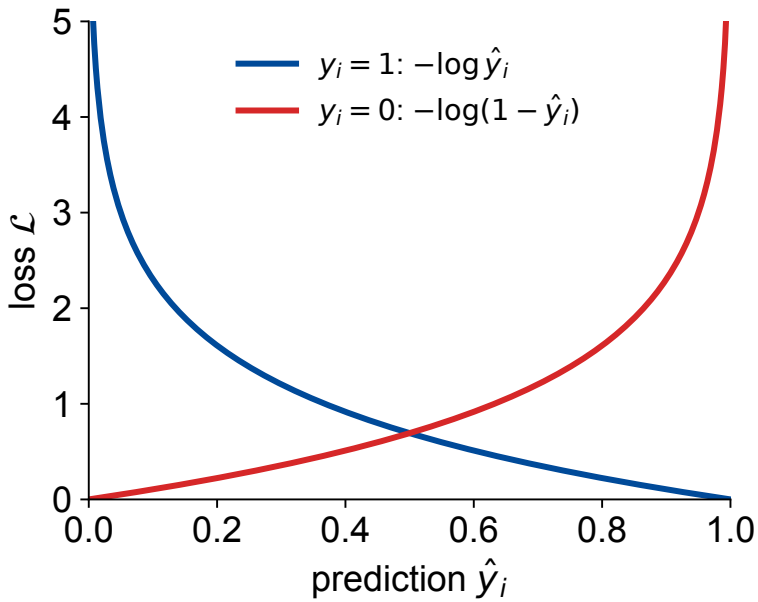
$$
\begin{aligned}
\hat{\boldsymbol{w}}_{ML} &= \underset{\boldsymbol{w}}{\operatorname{argmax}} \sum_{i=1}^{N} \log p_{\text{model}}(y_i \mid \boldsymbol{x}_i, \boldsymbol{w}) \\
&= \underset{\boldsymbol{w}}{\operatorname{argmax}} \sum_{i=1}^{N} \log \left[\hat{y}_i^{y_i} (1 - \hat{y}_i)^{(1-y_i)} \right] \\
&= \underset{\boldsymbol{w}}{\operatorname{argmin}} \sum_{i=1}^{N} \underbrace{-y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)}_{\text{Binary Cross Entropy Loss } \mathcal{L}(\hat{y}_i, y_i)}
\end{aligned}
$$

- In ML, we use the more general term “loss function” rather than “error function”
- Interpretation: We minimize the dissimilarity between the empirical data distribution p_{data} (defined by the training set) and the model distribution p_{model}

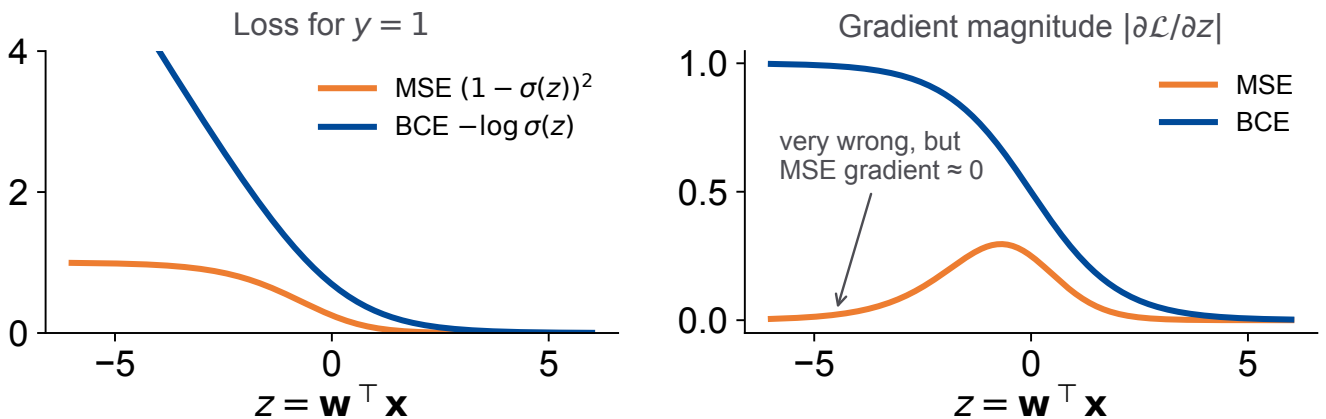
Binary Cross Entropy Loss

$$\mathcal{L}(\hat{y}_i, y_i) = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)$$
$$= \begin{cases} -\log \hat{y}_i & \text{if } y_i = 1 \\ -\log(1 - \hat{y}_i) & \text{if } y_i = 0 \end{cases}$$

- For $y_i = 1$ the loss $\mathcal{L}$ is minimized if $\hat{y}_i = 1$
- For $y_i = 0$ the loss $\mathcal{L}$ is minimized if $\hat{y}_i = 0$
- Thus, $\mathcal{L}$ is minimal if $\hat{y}_i = y_i$
- Confident wrong predictions are punished heavily
- Can be extended to > 2 classes (later in the lecture)



Why Not Squared Error for Classification?



- With a sigmoid output, the squared error has **vanishing gradients** for confident wrong predictions ($z \ll 0$ for $y = 1$) $\rightarrow$ learning gets stuck
- BCE: gradient $\hat{y} - y$ stays large as long as the prediction is wrong; BCE is convex in $\boldsymbol{w}$, MSE with sigmoid is not

Cross Entropy vs. Maximum Likelihood

$$\hat{\boldsymbol{w}}_{ML} = \underset{\boldsymbol{w}}{\operatorname{argmin}} \sum_{i=1}^{N} \underbrace{-y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)}_{\text{Binary Cross Entropy Loss } \mathcal{L}(\hat{y}_i, y_i)}$$

with $\hat{y} = f_{\boldsymbol{w}}(\boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x})$ and $\sigma(z) = \dfrac{1}{1 + e^{-z}}$

- **We maximize the probability of our classifier by minimizing the cross-entropy**
- In contrast to linear regression, the loss $\mathcal{L}(\hat{y}_i, y_i)$ is **not quadratic** in w
- No closed-form solution: we must apply iterative gradient-based optimization

Gradient of the BCE Loss

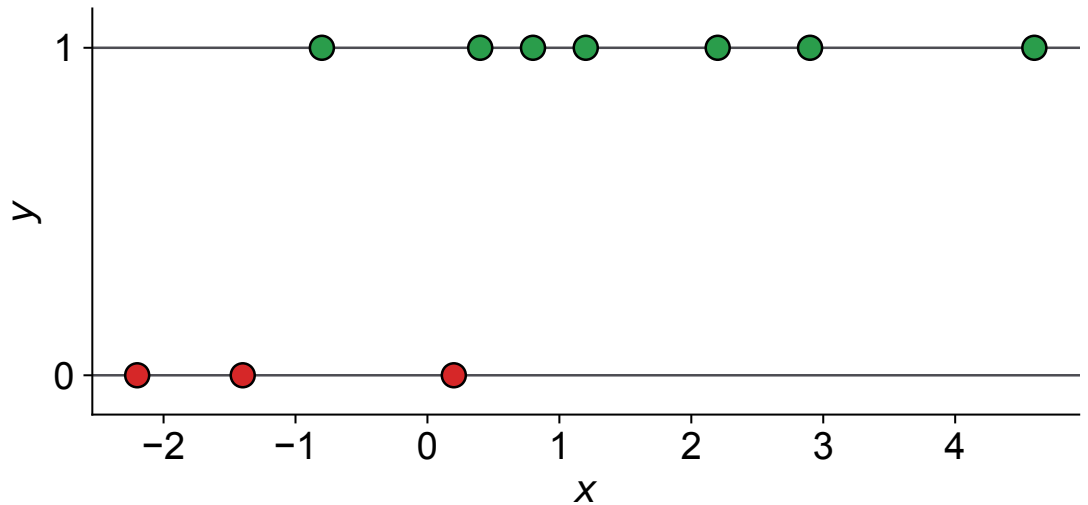
Chain rule with $z = \boldsymbol{w}^\top \boldsymbol{x},\ \hat{y} = \sigma(z),\ \sigma'(z) = \sigma(z)(1-\sigma(z)) = \hat{y}(1-\hat{y})$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}) \qquad \frac{\partial z}{\partial \boldsymbol{w}} = \boldsymbol{x}$$

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\hat{y}_i, y_i) = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial \boldsymbol{w}} = (\hat{y}_i - y_i)\boldsymbol{x}_i$$

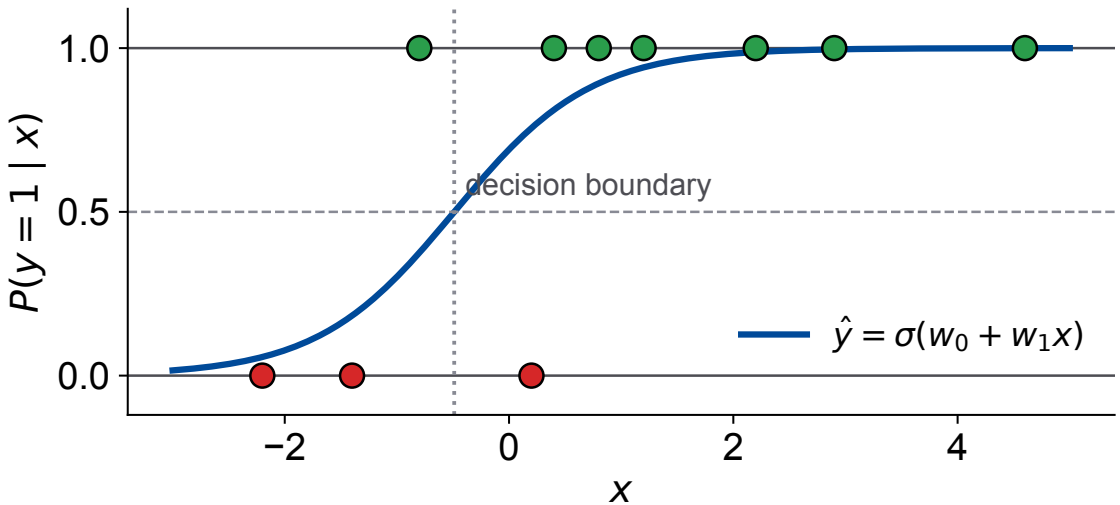
- Same form as for linear regression (up to the factor 2) – only $\hat{y}$ differs!
- Luckily, here the gradient is simple – but not for more complex models such as deep neural networks
- How can we **efficiently** compute gradients in the general case? → 3.5

Simple Example in 1D



- Positive class ($y = 1$) and negative class ($y = 0$), one feature x

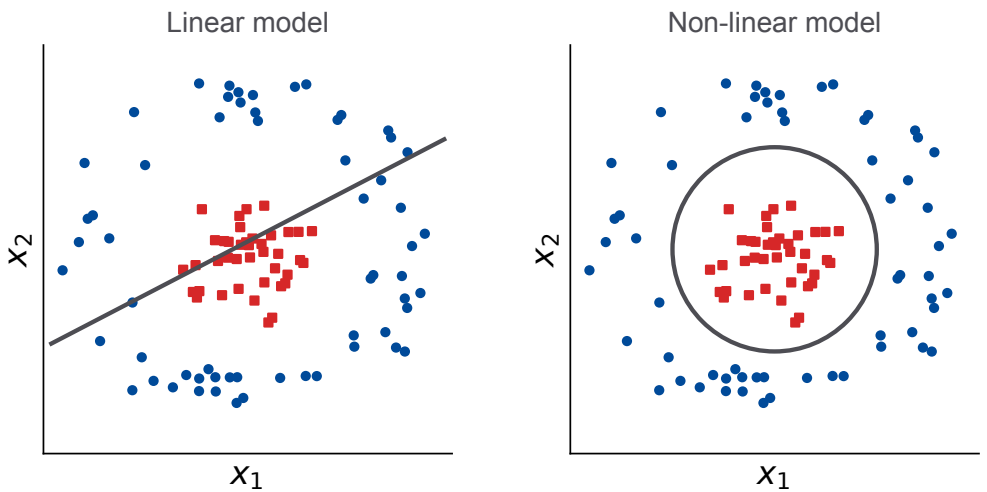
Simple Example in 1D



- Logistic regression model (fitted with gradient descent) to make predictions
- Decision boundary where $\hat{y} = 0.5$

Non-Linearity

- Activation functions add an element of non-linearity
- But: a single neuron still has a **linear** decision boundary
- Non-linear problems (right) need non-linear models
- → Stack several neurons with non-linear activations: multi-layer networks (lecture 4)



UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

3.5 Computation Graphs



Computation Graphs

- **Decompose** complex computations into sequence of atomic assignments
- We call this sequence of assignments a **computation graph** or **source code**
- The **forward pass** takes a training point (**x**, y) as input and computes a loss, e.g.:

$$\mathcal{L} = -\log p_{\text{model}}(y \mid \boldsymbol{x}, \boldsymbol{w})$$

- As we will see, gradients $\nabla_{\boldsymbol{w}} \mathcal{L}$ can be computed using a **backward pass**
- Both, the forward pass and the backward pass are **efficient** due to the use of dynamic programming, i.e., storing and reusing intermediate results
- This decomposition and reuse of computation is key to the success of the **backpropagation algorithm**, the primary workhorse of deep learning

Example Computation Graph

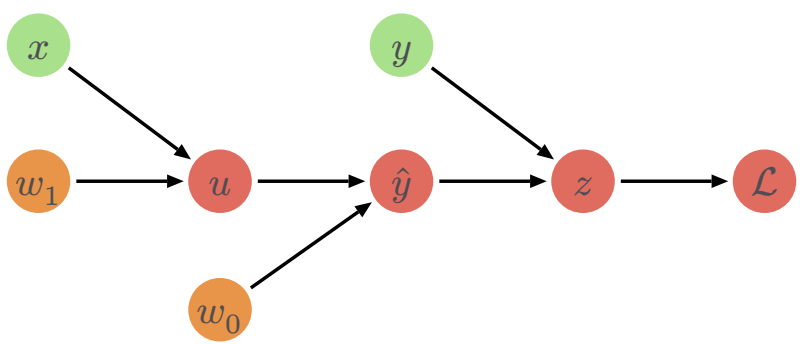
- A **computation graph** has three kinds of nodes:
  - Input nodes
  - Parameter nodes
  - Compute nodes
- **Example:** Linear Regression

(1) $u = w_1 x$

(2) $\hat{y} = w_0 + u$

(3) $z = \hat{y} - y$

(4) $\mathcal{L} = z^2$



Levels of Granularity

- Same function as before but different graph:

  (1) $\hat{y} = w_0 + w_1 x$

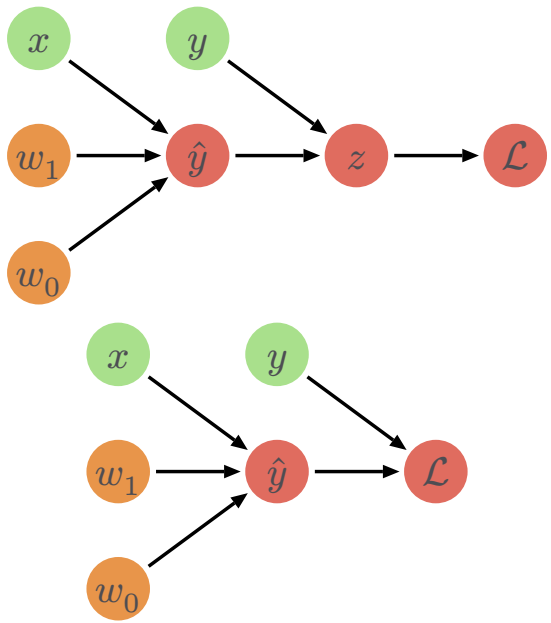
  (2) $z = \hat{y} - y$

  (3) $\mathcal{L} = z^2$

- Yet another graph for the same function:

  (1) $\hat{y} = w_0 + w_1 x$

  (2) $\mathcal{L} = (\hat{y} - y)^2$



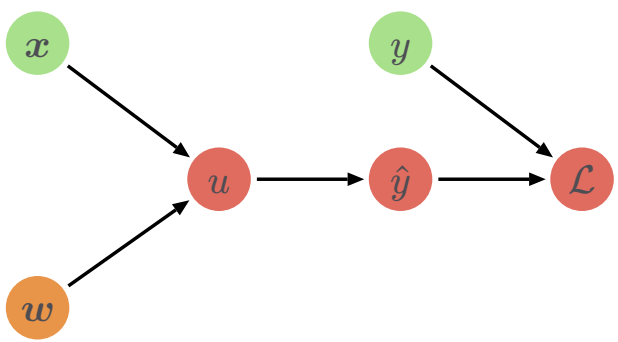
Example Computation Graph

- A **computation graph** has three kinds of nodes:
  - Input nodes
  - Parameter nodes
  - Compute nodes
- **Example:** *Logistic* Regression

(1) $u = \boldsymbol{w}^\top \boldsymbol{x}$

(2) $\hat{y} = \sigma(u)$

(3) $\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$



Preview: The Backward Pass

Linear regression graph with $x = 2$, $y = 3$, $w_0 = 1$, $w_1 = 0.5$:

Node	Forward pass (values)	Backward pass (gradients, chain rule)
$\mathcal{L} = z^2$	$\mathcal{L} = 1$	$\partial\mathcal{L}/\partial\mathcal{L} = 1$
$z = \hat{y} - y$	$z = -1$	$\partial\mathcal{L}/\partial z = 2z = -2$
$\hat{y} = w_0 + u$	$\hat{y} = 2$	$\partial\mathcal{L}/\partial\hat{y} = \partial\mathcal{L}/\partial z \cdot 1 = -2$
$u = w_1 x$	$u = 1$	$\partial\mathcal{L}/\partial u = \partial\mathcal{L}/\partial\hat{y} \cdot 1 = -2$
w_0, w_1		$\partial\mathcal{L}/\partial w_0 = -2, \quad \partial\mathcal{L}/\partial w_1 = \partial\mathcal{L}/\partial u \cdot x = -4$

- Forward: top to bottom of the assignments; backward: in reverse order, reusing stored values → lecture 4

Computation Graphs in PyTorch: Autograd

```python
import torch
x, y = torch.tensor(2.0), torch.tensor(3.0)
w0 = torch.tensor(1.0, requires_grad=True)
w1 = torch.tensor(0.5, requires_grad=True)

u = w1 * x              # forward pass:
y_hat = w0 + u          # builds the graph
z = y_hat - y
loss = z ** 2

loss.backward()         # backward pass
print(w0.grad, w1.grad)
# tensor(-2.) tensor(-4.)
```

- PyTorch records the computation graph during the forward pass
- `backward()` computes all gradients automatically
- Same numbers as on the previous slide
- More in exercise 02 (`02_pytorch_intro.ipynb`)

Summary and Outlook

Take-home messages

- Linear regression: least squares, closed form (normal equation) or gradient descent
- Gradient descent: step against the gradient; learning rate matters; (mini-batch) SGD for large data
- Perceptron: linear decision boundary, simple update rule, fails on XOR
- Logistic regression: sigmoid output = probability, trained by maximum likelihood = binary cross-entropy
- Computation graphs: forward pass computes the loss, backward pass the gradients

Next lecture: Multi-Layer Neural Networks

- XOR and linear separability
- Multi-layer perceptrons: hidden layers, non-linear activations
- Expressivity of MLPs
- Training MLPs: backpropagation